# Overview

# Mobile Build Blueprint - webapp-main

## Resolved source

Exact commit SHA: 368cec4aff9f293a2564eb6c35f3d9045b343c2c. This is the latest main-branch commit at or before the requested cutoff; later commits were excluded. Repository: kashyapempiricinfotech-sys/webapp-main. Analysis was code-only: the app was not run and no live endpoint was called.

## Build recommendation

Proceed with React Native plus Expo and preserve the existing TypeScript service/domain layer where contracts are valid (inferred recommendation). Treat the first two sprints as a contract-and-auth remediation gate, because the checked-in client and server do not currently agree on settings or push-installation endpoints and the JWT flow is not production-ready. [source: package.json:1-27; src/services/settings.ts:1-5; src/services/notifications.ts:1-8; server/routes/notifications.ts:1-5; server/routes/auth.ts:1-12; server/middleware/requireAuth.ts:1-13]

### Architecture

Maps runtime layers, active versus dormant business modules, every server endpoint, auth generations, and third-party integrations.

### Screen Inventory

Lists all six filesystem screens and four reusable component abstractions, including direct reuse counts and feature/role gates.

### Mobile Stack

Compares two real options in each required category, records current published plan pricing, and makes a build-ready recommendation.

### User Flow

Separates the currently wired interactions from the proposed mobile navigation flow and marks every proposed transition as inferred.

### Data Model

Documents the three Prisma models, relationships, TypeScript contract differences, and the fact that persistence is declared but unwired.

## Delivery Plan

Groups the mobile build into four dependency-ordered phases spanning Sprints 1 through 6, matching the Jira work.

## Risks & Open Questions

Prioritizes contract, auth, payment/deep-link, persistence, observability, and release questions that must be resolved before shipping.

## Secrets found (review)

Points to credential-shaped material by file and line only; no value is reproduced.

# Architecture

# Architecture

## System shape

The UI is a Next 15.4 and React 19 filesystem-routed app written in TypeScript. A separate Express 5 process mounts JSON APIs on port 3001. Relative fetch calls in the browser code do not define how the Next and Express origins are joined (inferred deployment gap). [source: package.json:1-27; server/index.ts:1-26; src/services/auth.ts:3-8; src/services/invoices.ts:4-12]
The declared persistence layer is PostgreSQL through Prisma, located in a nested package. No checked-in route imports a Prisma client, so the schema is a design declaration rather than active persistence. [source: packages/data/prisma/schema.prisma:1-32; server/routes/auth.ts:1-12; server/routes/invoices.ts:1-10; server/routes/notifications.ts:1-5]

## Business logic modules

Authentication - active. The login service posts credentials to /api/auth/login and adds bearer headers. Server login derives role from the email suffix, does not validate the password, signs a JWT without an explicit expiry, and refresh returns a token-shaped constant. [source: src/services/auth.ts:1-11; server/routes/auth.ts:1-12]
Invoices and checkout - partly active. Client functions list invoices and request checkout; the server returns an empty list and creates a Stripe Checkout Session for a fixed amount. The invoices screen itself renders hard-coded demo data and does not call the service. [source: src/services/invoices.ts:1-13; server/routes/invoices.ts:1-10; src/integrations/stripe.ts:1-12; app/invoices/page.tsx:1-10]
Notification preferences - server active, client contract mismatched. Express exposes GET/PUT /api/notifications/preferences, while the settings screen calls PUT /api/settings. [source: server/routes/notifications.ts:1-5; src/services/settings.ts:1-5; app/settings/page.tsx:1-7]
Push installation - client-only. The client posts to /api/notifications/installations, but no matching server handler is registered. The notifications-center screen is separately gated by FEATURE_PUSH. [source: src/services/notifications.ts:1-8; server/index.ts:13-23; app/notifications-center/page.tsx:1-6]
Reporting aggregation - dormant. reportingAggregator is explicitly documented as unwired and has no import site. [source: src/services/reportingAggregator.ts:1-6]

# API endpoint inventory

GET /api/health - public health response from the standalone handler. [source: api/health.ts:1-3]

POST /api/auth/login - public JWT issuance; role is derived from an email suffix and password validation is absent. [source: server/index.ts:16; server/routes/auth.ts:5-9]

POST /api/auth/refresh - public token refresh stub; no refresh credential is read. [source: server/routes/auth.ts:11]

GET /api/invoices - bearer JWT required by mount middleware; returns an empty array. [source: server/index.ts:17; server/routes/invoices.ts:5]

POST /api/invoices/:invoiceId/checkout - bearer JWT required; creates Stripe Checkout and returns its URL. [source: server/index.ts:17; server/routes/invoices.ts:6-9; src/integrations/stripe.ts:5-11]

GET /api/notifications/preferences - bearer JWT required; returns paymentReminders. [source: server/index.ts:18; server/routes/notifications.ts:4]

PUT /api/notifications/preferences - bearer JWT required; returns the submitted boolean. [source: server/index.ts:18; server/routes/notifications.ts:5]

GET /api/legacy/profile - legacy cookie session; returns a nullable legacy user id. [source: server/index.ts:19; server/routes/legacy.ts:1-6; server/auth/legacySession.ts:1-8]

POST /api/legacy/logout - legacy cookie session; clears the session and returns 204. [source: server/routes/legacy.ts:7]

GET /api/catalog - public manifest route; returns one starter item. [source: server/index.ts:21-23; server/routes/manifest.ts:5-8]

GET /api/audit-events - bearer JWT required; returns an empty array. It does not enforce an admin role (inferred authorization gap). [source: server/index.ts:21-23; server/routes/manifest.ts:5-8; server/middleware/requireAuth.ts:4-13]

# Client/server contract gaps

PUT /api/settings exists only in src/services/settings.ts; no registered server route matches it. POST /api/notifications/installations exists only in src/services/notifications.ts; no registered server route matches it. These are build blockers, not optional mobile adaptations. [source: src/services/settings.ts:3-5; src/services/notifications.ts:5-7; server/index.ts:13-23]

# Authentication flows

Current path - bearer JWT. signIn posts email/password, receives token/user, and callers are expected to use Authorization: Bearer. No client token storage, bootstrap, logout, expiry handling, or refresh orchestration exists. [source: src/services/auth.ts:1-11]
Server verification - requireAuth strips the Bearer prefix and verifies with JWT_PUBLIC_KEY. Login signs without specifying an asymmetric algorithm, while it uses JWT_PRIVATE_KEY; the default/fallback pairing is inconsistent for production JWT verification (inferred from jsonwebtoken defaults and code shape). [source: server/routes/auth.ts:1-12; server/middleware/requireAuth.ts:1-13]
Legacy path - cookie-session remains mounted for profile and logout. The cookie is httpOnly, but secure and sameSite options are not set in this cutoff commit. [source: server/auth/legacySession.ts:1-8; server/routes/legacy.ts:1-7]

# Third-party integrations

Stripe - active through the authenticated invoice checkout route. Success and cancel callbacks use the custom webapp:// scheme. [source: server/routes/invoices.ts:1-10; src/integrations/stripe.ts:1-12]
PostHog - startAnalytics is called during Express startup. trackInvoiceOpened is defined but not imported elsewhere, so event capture is dormant. [source: server/index.ts:1-14; src/integrations/posthog.ts:1-9]
Sentry - startup is called, but initialization is a no-op unless SENTRY_ENABLED is true. [source: server/index.ts:1-14; src/integrations/sentry.ts:1-7; .env.example:4]
Cloud bootstrap configuration - a credential-shaped identifier is hard-coded and not imported anywhere in the repository. See Secrets found (review); value withheld. [source: src/config/cloud.ts:1-5]

# Screen Inventory

# Screen Inventory

## Screens

/login - Sign-in screen (inferred route from app directory). Uses PrimaryCta and calls signIn with demo credentials. Reusable shell is not used. [source: app/login/page.tsx:1-7]

/dashboard - Account snapshot landing screen (inferred route). Uses AppShell and PrimaryCta. The Review invoices button has no navigation handler. [source: app/dashboard/page.tsx:1-5]

/invoices - Invoice list/create surface (inferred route). Uses AppShell, InvoiceCard, PrimaryCta through withTelemetry, and a hard-coded demo invoice. The Create invoice button has no handler. [source: app/invoices/page.tsx:1-10]

/settings - Reminder settings screen (inferred route). Uses AppShell and PrimaryCta; save calls the unmatched /api/settings contract with a literal placeholder session token. [source: app/settings/page.tsx:1-7; src/services/settings.ts:1-5]

/notifications-center - Feature-gated notification center (inferred route). Returns no UI unless FEATURE_PUSH equals true. Uses AppShell. [source: app/notifications-center/page.tsx:1-6]

/admin-reports - Role-gated revenue report (inferred route). Non-admin users receive a Forbidden paragraph; admins receive AppShell content. The source of the user prop is not shown (inferred integration gap). [source: app/admin-reports/page.tsx:1-7]

## Reusable components and direct reuse

AppShell - page header/main layout. Directly used by five screens: dashboard, invoices, settings, notifications-center, and admin-reports. [source: src/components/AppShell.tsx:1-5; app/dashboard/page.tsx:1-5; app/invoices/page.tsx:1-10; app/settings/page.tsx:1-7; app/notifications-center/page.tsx:1-6; app/admin-reports/page.tsx:1-7]

PrimaryCta - alias of ActionButton. Directly imported by four screens: login, dashboard, invoices, and settings. [source: src/components/index.ts:1; src/components/ActionButton.tsx:1-5; app/login/page.tsx:1-7; app/dashboard/page.tsx:1-5; app/invoices/page.tsx:1-10; app/settings/page.tsx:1-7]

InvoiceCard - invoice number/status card. Used by one screen: invoices. [source: src/components/InvoiceCard.tsx:1-5; app/invoices/page.tsx:1-10]

withTelemetry - higher-order wrapper that adds a data-analytics-component marker. Instantiated once around PrimaryCta on the invoices screen. [source: src/components/withTelemetry.tsx:1-7; app/invoices/page.tsx:1-10]

## Mobile reuse boundary

Types and pure service/domain logic can be adapted, but DOM elements, Next pages, and web fetch assumptions cannot be reused as native UI. AppShell, ActionButton, InvoiceCard, and withTelemetry should be reimplemented with React Native primitives while preserving their semantic roles (inferred recommendation). [source: src/components/ActionButton.tsx:1-5; src/components/AppShell.tsx:1-5; src/components/InvoiceCard.tsx:1-5; src/components/withTelemetry.tsx:1-7]

# Mobile Stack

# Mobile Stack

## Decision

Recommend React Native with Expo Router, Expo Modules, TypeScript, TanStack Query, and native secure storage (inferred recommendation). The existing code is already React/TypeScript, React Native's own guidance recommends using a framework such as Expo for new apps, and the required custom work is concentrated in API contracts, auth, deep links, push, and payments rather than rendering algorithms. [source: package.json:1-27; src/types.ts:1-16; React Native overview]

## Cross-platform framework options

React Native plus Expo - Open source tier, $0/month license. Best fit because types and service logic remain TypeScript and the team can preserve React mental models. Web components still require native rewrites. Recommended. [source: package.json:1-27; React Native license; Expo pricing]

Flutter - Open source tier, $0/month license. Strong multi-platform rendering and tooling, but requires a Dart rewrite of all client code and loses the direct TypeScript/React alignment. Keep as the alternative if the organization already has a Flutter team. [source: package.json:1-27; Flutter license]

## Native bridge options

Expo Modules API - Open source tier, $0/month license. Swift/Kotlin modules support the React Native New Architecture with low boilerplate. Use Expo libraries first for secure storage, deep linking, notifications, and web-browser checkout; add a custom module only when a required native SDK is unavailable. Recommended. [source: src/integrations/stripe.ts:5-11; src/services/notifications.ts:1-8; Expo Modules API; Expo pricing]

React Native Turbo Native Modules - Open source tier, $0/month license. Offers lower-level control and C++ access, but increases iOS/Android build ownership and native maintenance. Reserve for an SDK or performance case Expo Modules cannot cover. [source: React Native license; Expo Modules API]

## Testing tool options

Firebase Test Lab - Spark plan, $0/month, with published daily no-cost quotas. Pair Vitest (already configured) with React Native Testing Library and Maestro smoke tests, then run a small device matrix in Test Lab. Cheapest starting point; physical-device depth is quota-limited. Recommended for Sprints 1-4. [source: package.json:4-9,21-26; Firebase Test Lab pricing]

Sauce Labs Real Device Cloud - 1 parallel, $249/month when billed month to month. Adds broad real iOS/Android device coverage, video, logs, and unlimited automated minutes at the quoted tier. Buy for Sprints 5-6 or release windows if the device matrix justifies it. [source: Sauce Labs pricing]

## CI/CD pipeline options

Expo EAS - Starter plan, $19/month plus usage and $45 of included build credit. Use pull-request checks in GitHub, EAS internal builds on main, device smoke tests, then signed store submissions from protected release workflows. Best fit with the recommended framework. [source: Expo pricing; Expo plans]

Codemagic - Pay as you go tier, $0 base plus $0.095 per Mac mini M2 minute. A rough 1,000-minute month is about $95. It is framework-neutral and gives more pipeline control, but needs more signing/build configuration. Prefer if the team rejects EAS or needs mixed mobile frameworks. [source: Codemagic pricing]

## Cost baseline

Recommended launch baseline: $19/month for EAS Starter plus $0 for framework, bridge, and Firebase Spark testing. Add about $8.25/month when amortizing the $99/year Apple Developer Program, and a one-time $25 Google Play registration fee. Sauce Labs is an optional $249/month release-phase upgrade; overages, taxes, labor, backend hosting, and

Stripe transaction fees are excluded. [source: Expo pricing; Firebase Test Lab pricing; Apple Developer Program; Google Play Console registration]

# Pipeline stages

Pull request (inferred) - dependency lock verification, TypeScript typecheck, Vitest, component tests, API contract tests, secret scanning, and no signed builds. [source: package.json:4-9]
Main branch (inferred) - EAS internal Android/iOS builds, installable artifact retention, device smoke flow, and preview telemetry using non-production keys. [source: src/integrations/posthog.ts:1-9; src/integrations/sentry.ts:1-7]
Release tag (inferred) - production signing, store metadata validation, deep-link tests for Stripe return paths, Sentry release upload if enabled, manual approval, staged rollout, and rollback notes. [source: src/integrations/stripe.ts:5-11; src/integrations/sentry.ts:1-7]

# Published pricing sources

React Native license | Flutter license | Expo pricing | Expo plans | Expo Modules API | Firebase Test Lab pricing | Sauce Labs pricing | Codemagic pricing | Apple Developer Program | Google Play Console registration

# Options at a glance

| Category | Option and tier | Published monthly cost | Decision |
|---|---|---|---|
| Framework | React Native + Expo - open source | $0 license | Recommended |
| Framework | Flutter - open source | $0 license | Alternative |
| Native bridge | Expo Modules API - open source | $0 license | Recommended |
| Native bridge | React Native TurboModules - open source | $0 license | Escape hatch |
| Testing | Firebase Test Lab - Spark | $0 | Start |
| Testing | Sauce Real Device Cloud - 1 parallel | $249 month to month | Release phase |
| CI/CD | Expo EAS - Starter | $19 plus usage | Recommended |
| CI/CD | Codemagic - Pay as you go | About $95 at 1,000 M2 minutes | Alternative |

# User Flow

# User Flow

## Current wiring

No screen-to-screen navigation is implemented in the checked-in pages. Dashboard and invoice CTAs have no handlers, and only login/settings perform service calls. The flow below is therefore the proposed mobile navigation (inferred), grounded in the existing screen and API files. [source: app/dashboard/page.tsx:1-5; app/invoices/page.tsx:1-10; app/login/page.tsx:1-7; app/settings/page.tsx:1-7]

### Step 1 - App launch and session bootstrap (inferred)

Read the token from Keychain/Keystore. If absent or invalid, open Sign in. If present, validate/refresh before entering the authenticated navigator. Token storage and bootstrap do not exist in the web source and must be added. [source: src/services/auth.ts:1-11; server/routes/auth.ts:1-12; server/middleware/requireAuth.ts:1-13]

### Step 2 - Sign in

Submit email/password to POST /api/auth/login, persist the returned token/user securely, and route to Dashboard. The server contract must first gain real credential validation, token expiry, and a production refresh flow. [source: app/login/page.tsx:1-7; src/services/auth.ts:3-8; server/routes/auth.ts:5-11]

### Step 3 - Dashboard to invoices (inferred)

Dashboard becomes the authenticated home. Review invoices navigates to Invoices. This transition is not wired in the web page. [source: app/dashboard/page.tsx:1-5]

### Step 4 - Invoice list and checkout (inferred)

Invoices loads GET /api/invoices, selects an invoice, and requests POST /api/invoices/:invoiceId/checkout. Open the returned Checkout URL in a secure system browser, then handle the webapp:// success/cancel deep link. Add a mobile invoice-detail/result screen because no matching route currently exists. [source: src/services/invoices.ts:1-13; server/routes/invoices.ts:1-10; src/integrations/stripe.ts:5-11; app/invoices/page.tsx:1-10]

### Step 5 - Settings and notification preferences (inferred)

Settings reads and writes /api/notifications/preferences after the client/server contract is unified. Keep Notifications Center hidden unless the rollout flag is enabled, but do not let a hidden UI imply the server preference endpoints are disabled. [source: app/settings/page.tsx:1-7; src/services/settings.ts:1-5; server/routes/notifications.ts:1-5; app/notifications-center/page.tsx:1-6]

## Step 6 - Push enrollment (inferred)

Request OS permission after user intent, obtain a platform token, and register it against an authenticated server endpoint. The client path exists, but the server installation endpoint does not. [source: src/services/notifications.ts:1-8; server/index.ts:13-23]

## Step 7 - Admin reports (inferred)

Show the Admin reports route only for admin users and enforce the role again on every supporting API. The screen has a role check, but no supporting revenue endpoint or server-side admin authorization appears in the repository. [source: app/admin-reports/page.tsx:1-7; server/routes/manifest.ts:5-8; server/middleware/requireAuth.ts:4-13]

## Step 8 - Sign out and legacy isolation (inferred)

Clear native secure storage and return to Sign in. Do not make the mobile app depend on cookie-session; keep /api/legacy/profile and /api/legacy/logout isolated for old clients until a retirement date is agreed. [source: server/routes/legacy.ts:1-7; server/auth/legacySession.ts:1-8]

# Data Model

# Data Model

## Declared database

PostgreSQL is declared through Prisma. The repository contains no migration files, generated client usage, or route-level database imports, so persistence behavior is not implemented in this snapshot. [source: packages/data/prisma/schema.prisma:1-32; server/routes/auth.ts:1-12; server/routes/invoices.ts:1-10; server/routes/notifications.ts:1-5]

### User

Fields: id (primary key), email (unique), role. Relationships: one-to-many invoices and optional one-to-one NotificationSetting. Role is a free-form database string, while the TypeScript SessionUser narrows it to user or admin. [source: packages/data/prisma/schema.prisma:10-16; src/types.ts:1-8]

### Invoice

Fields: id, ownerId, number, totalMinor, currency, status. ownerId references User.id. The TypeScript Invoice narrows currency to USD or INR and status to draft, open, or paid, but Prisma stores both as unrestricted strings. [source: packages/data/prisma/schema.prisma:18-26; src/types.ts:10-16]

### NotificationSetting

Fields: userId (primary key) and paymentReminders (default true). userId is also the one-to-one relation to User.id. [source: packages/data/prisma/schema.prisma:28-32]

## Mobile data contracts (inferred)

Generate a versioned API schema for SessionUser, Invoice, NotificationSetting, login/refresh, checkout, and push installation. Keep currency/status as shared enums or database constraints, add timestamps/version fields, and make offline caches disposable rather than authoritative. These fields are recommended because the current TypeScript and Prisma definitions can drift. [source: packages/data/prisma/schema.prisma:10-32; src/types.ts:1-16]

# Delivery Plan

# Delivery Plan

## Phase 1 - foundation, API contracts, and authentication

Sprints 1-2. Establish the React Native/Expo workspace, navigation shell, environment handling, typed API client, secure token storage, production JWT/refresh contract, legacy-session isolation, contract tests, and secret remediation. This phase blocks all authenticated product work. [source: package.json:1-27; src/services/auth.ts:1-11; server/routes/auth.ts:1-12; server/middleware/requireAuth.ts:1-13; server/auth/legacySession.ts:1-8]
Done means both platforms build in internal distribution, login/refresh/logout survive restart and expiry, role data is trustworthy, client/server contract tests pass, and no credential-shaped literal is shipped. [source: src/config/cloud.ts:1-5; src/services/notifications.ts:1-8; server/routes/auth.ts:1-12]

## Phase 2 - core navigation, invoices, and checkout

Sprints 3-4. Build Dashboard, Invoices, invoice detail/result, native components, list/loading/error states, typed invoice API integration, Stripe Checkout browser handoff, and success/cancel deep-link handling. [source: app/dashboard/page.tsx:1-5; app/invoices/page.tsx:1-10; src/services/invoices.ts:1-13; server/routes/invoices.ts:1-10; src/integrations/stripe.ts:1-12]
Done means the seeded fixture path is replaced by API data, checkout deep links return to a defined screen, duplicate taps are safe, and Android/iOS end-to-end smoke tests cover success, cancel, and failure. [source: app/invoices/page.tsx:5-10; src/integrations/stripe.ts:5-11]

## Phase 3 - settings, notifications, analytics, and admin

Sprints 4-5. Unify preference endpoints, implement push installation server support, permission UX, feature gating, notifications center, PostHog event taxonomy, Sentry enablement, and admin report navigation/API authorization. [source: src/services/settings.ts:1-5; src/services/notifications.ts:1-8; server/routes/notifications.ts:1-5; app/notifications-center/page.tsx:1-6; src/integrations/posthog.ts:1-9; src/integrations/sentry.ts:1-7; app/admin-reports/page.tsx:1-7]
Done means permission denial is graceful, registration and preference APIs are contract-tested, analytics excludes sensitive data, Sentry is environment-controlled, and admin authorization is enforced server-side. [source: server/middleware/requireAuth.ts:4-13; server/routes/manifest.ts:5-8]

# Phase 4 - quality, hardening, and release readiness

Sprints 5-6. Complete device-matrix testing, accessibility, performance, offline/error behavior, privacy disclosures, store assets, signing, CI/CD gates, staged rollout, rollback, and legacy retirement evidence. [source: package.json:4-9; server/routes/legacy.ts:1-7]
Done means automated acceptance gates pass, no open P0/P1 security or contract risks remain, store submissions are reproducible from protected workflows, observability is verified in a non-production release, and every issue remains To Do until the team intentionally starts it. [source: package.json:4-9; src/integrations/posthog.ts:1-9; src/integrations/sentry.ts:1-7]

# Risks & Open Questions

# Risks & Open Questions

## P0 - credential-shaped material

Several files contain hard-coded or fallback credential/token-shaped literals. Values are withheld; see Secrets found (review). Rotate/revoke anything real, remove client-visible proof material, and add repository/history/CI secret scanning before mobile code is distributed. [source: src/config/cloud.ts:2; src/services/notifications.ts:3; server/routes/auth.ts:7,11; server/middleware/requireAuth.ts:8; server/auth/legacySession.ts:6]

## P0 - authentication is a fixture, not a production protocol

Password validation is absent, role is inferred from an email suffix, token expiry is unspecified, refresh ignores input, and signing/verification configuration is inconsistent (inferred). Decide algorithm, key ownership/rotation, claims, TTL, refresh rotation/revocation, and logout semantics. [source: server/routes/auth.ts:5-11; server/middleware/requireAuth.ts:4-13]

## P0 - client/server contract mismatches

Choose the canonical notification-preference path and implement the push-installation endpoint or remove the client call. Add generated/shared contracts and tests. [source: src/services/settings.ts:3-5; src/services/notifications.ts:5-7; server/routes/notifications.ts:1-5; server/index.ts:13-23]

## P1 - payment return path has no destination screen

Stripe returns webapp://invoices/:invoiceId with success/cancel state, but there is no invoice-detail filesystem route. Decide custom scheme plus universal/app links, replay behavior, and whether v1 retains hosted Checkout or adds native PaymentSheet. [source: src/integrations/stripe.ts:5-11; app/invoices/page.tsx:1-10]

## P1 - persistence is not wired

Confirm the owning backend/database, migrations, seed strategy, timestamps, row-level authorization, and enum constraints. Current routes return fixtures and do not use Prisma. [source: packages/data/prisma/schema.prisma:1-32; server/routes/invoices.ts:5-9; server/routes/notifications.ts:4-5]

# P1 - admin enforcement and data source are incomplete

Where does the AdminReportsPage user prop come from, what API supplies revenue data, and where is admin authorization enforced on the server? Bearer authentication alone does not enforce role. [source: app/admin-reports/page.tsx:1-7; server/middleware/requireAuth.ts:4-13; server/routes/manifest.ts:5-8]

# P1 - observability lifecycle

PostHog startup is active but invoice event capture is unwired. Sentry is dormant behind a flag. Define mobile-safe event names, consent, PII redaction, release/environment tags, and kill switches. [source: server/index.ts:11-14; src/integrations/posthog.ts:1-9; src/integrations/sentry.ts:1-7]

# P2 - deployment topology

What is the production API base URL, how are Next and Express deployed, and what CORS/versioning rules apply to native clients? Relative fetch paths and a separate port do not answer this (inferred). [source: server/index.ts:25; src/services/auth.ts:4; src/services/invoices.ts:5,11; src/services/settings.ts:4]

# P2 - product scope and rollout

Is notifications center part of launch, is invoice creation actually in scope, and when can legacy cookie endpoints be removed? Current UI labels imply flows that the code does not implement. [source: app/notifications-center/page.tsx:1-6; app/invoices/page.tsx:8-10; server/routes/legacy.ts:1-7]

# Secrets found (review)

# Secrets found (review)

## Handling rule

Potential secret, key, token, or credential values are intentionally omitted. Review the referenced locations and repository history; rotate/revoke any value that is real before distributing a mobile build.

### src/config/cloud.ts:2

Hard-coded cloud access identifier with credential shape. Value withheld.

### src/services/notifications.ts:3

Long embedded bootstrap/proof literal in client-consumable code. Value withheld.

### server/routes/auth.ts:7

Fallback JWT signing credential. Value withheld.

### server/routes/auth.ts:11

Hard-coded token-shaped refresh response. Value withheld.

### server/middleware/requireAuth.ts:8

Fallback JWT verification credential. Value withheld.

### server/auth/legacySession.ts:6

Fallback cookie-session signing credential. Value withheld.

### .env.example:1-3

Credential/key environment variable placeholders. Values not reproduced; verify that real values were never committed in history.

### app/login/page.tsx:6

Demo credential literals are passed into signIn. Values withheld; replace the fixture interaction before launch.

## app/settings/page.tsx:6

Token-shaped placeholder is passed to saveSettings. Value withheld; mobile must obtain credentials only from secure storage/session state.